

BOOLEANS

Yes/no switches

THE TYPE

Only ever one of two values

```
bool hasReceipt{false};  
bool isOverBudget{true};
```

Perfect for the yes/no questions an expense tracker needs to answer:
did we keep the receipt? are we over budget?

Explicit vs. direct

```
if (hasReceipt == true) {  
    std::println("Receipt on file.");  
} else {  
    std::println("No receipt - might get flagged in an audit.");  
}
```

```
if (isOverBudget) {    // no "== true" needed  
    std::println("Over budget this month!");  
}
```

Comparing a bool to true/false explicitly works, but it's redundant - the bool itself already is the yes/no answer.

SIZE

The smallest type we've seen

```
std::println("sizeof(bool): {}", sizeof(bool));
```

A `bool` only needs to distinguish two states, but C++ still gives it a full byte - there's no smaller addressable unit of memory to use.

PRINTING IT

Words by default, digits on request

```
std::println("hasReceipt : {}", hasReceipt); // true
std::println("hasReceipt : {:d}", hasReceipt); // 1
```

`std::println` formats a `bool` as the word `true` / `false` by default - `{:d}` instead prints the underlying integer: `1` for true, `0` for false.

DEBUG IT

Toggle it and watch the branch change

Breakpoint on the `if (isOverBudget)` line, then use your IDE's locals view to flip the value from `true` to `false` mid-run and step forward - watching the branch taken actually change confirms you understand which path a bool controls.